Introdução ao Servlet e Padrão MVC

Conteudista: Prof. Me. Alexander Gobbato Paulino Albuquerque
Revisão Textual: Prof. Claudio Brites

Objetivos da Unidade:

- Obter conhecimento sobre a funcionalidade do *Servlet* para a prática de programação *java* para *web*, permitindo dar início a criação de um projeto utilizando o padrão MVC;
- Adquirir conhecimentos para criar um projeto de CRUD.

Material Teórico

Material Complementar

Referências

Material Teórico

Arquitetura *Web* e Serviços

A cada Unidade que avançamos, vamos absorvendo mais conteúdo. Nesta Unidade, entenderemos como funciona o *Servlet*.

A programação *java* para *web* nos permite escrever aplicações profissionais robustas e seguras. Para isso, teremos que utilizar a tecnologia *Java Servlet* e *JavaServer Pages*.

- ***Servlets*:** são classes *Java* que atendem às requisições HTTP;
- **JSP:** é o código *Java* junto com HTML, eles são usados principalmente como interface visual para o cliente *web* (navegador).

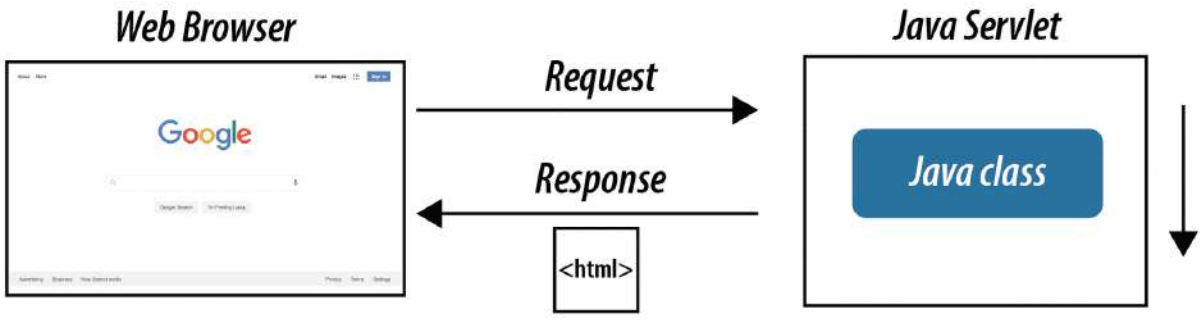


Figura 1
Fonte: Reprodução

O Servidor *Java* é o local onde ficam armazenados os *Servlets*, os JSPs e as classes de negócio da sua aplicação.

O Servidor *Java* atende às solicitações feitas a ele, invocando os recursos solicitados, como os *Servlets*, JSPs, HTMLs, imagens etc., assim como um *webserver*, porém estendendo a funcionalidade dos *webservers*, servindo as aplicações em *Java*.

Esses servidores são, geralmente, conhecidos como *Servlet Containers*, ou *Web Containers*.

Outros servidores mais completos que implementam toda a especificação J2EE podem ser encontrados. Alguns deles: *JBoss*, IBM *Web Sphere*, Bea *Web Logic*, *Oracle* OC4J, BES (*Borland*) e outros.

O *container Java* também é conhecido como servidor de aplicações *Java*.

Gerencia a execução de serviços *web Java*.

Disponibilizar infraestrutura necessária para esses serviços.

Permite a comunicação com outros componentes *Java* e aplicações.

Servidor de Aplicações – Funcionamento básico:

- Aguarda pelas *requests*;
- Repassa a *request* para o ***servlet* correto**;
- Recebe a *response* do ***servlet***;
- Devolve a *response* para o cliente que enviou a *request*.

Ou seja, ele é responsável por gerenciar as conexões (dentre outras coisas).

O servidor de aplicativos não faz só isso...

Dentre outras coisas...

Ele também cuida do **ciclo de vida do *servlet***:

- Quando necessário, inicia o *Servlet*;
- Coloca este *Servlet* no estado "ativo";
- Finaliza o *Servlet* quando não é mais necessário.

Para utilizar um *servlet*, precisamos saber as seguintes informações:

- **Passo 1:** configurar o *content type*;
- **Passo 2:** configurar o objeto para "escrever" no navegador;
- **Passo 3:** gerar o html que será exibido no navegador.

Como criar um projeto *servlet* no *eclipse*:

O processo para criação do projeto é bem simples, ao acessar o eclipse, vamos na opção *File→New→Dynamic Web Project.*

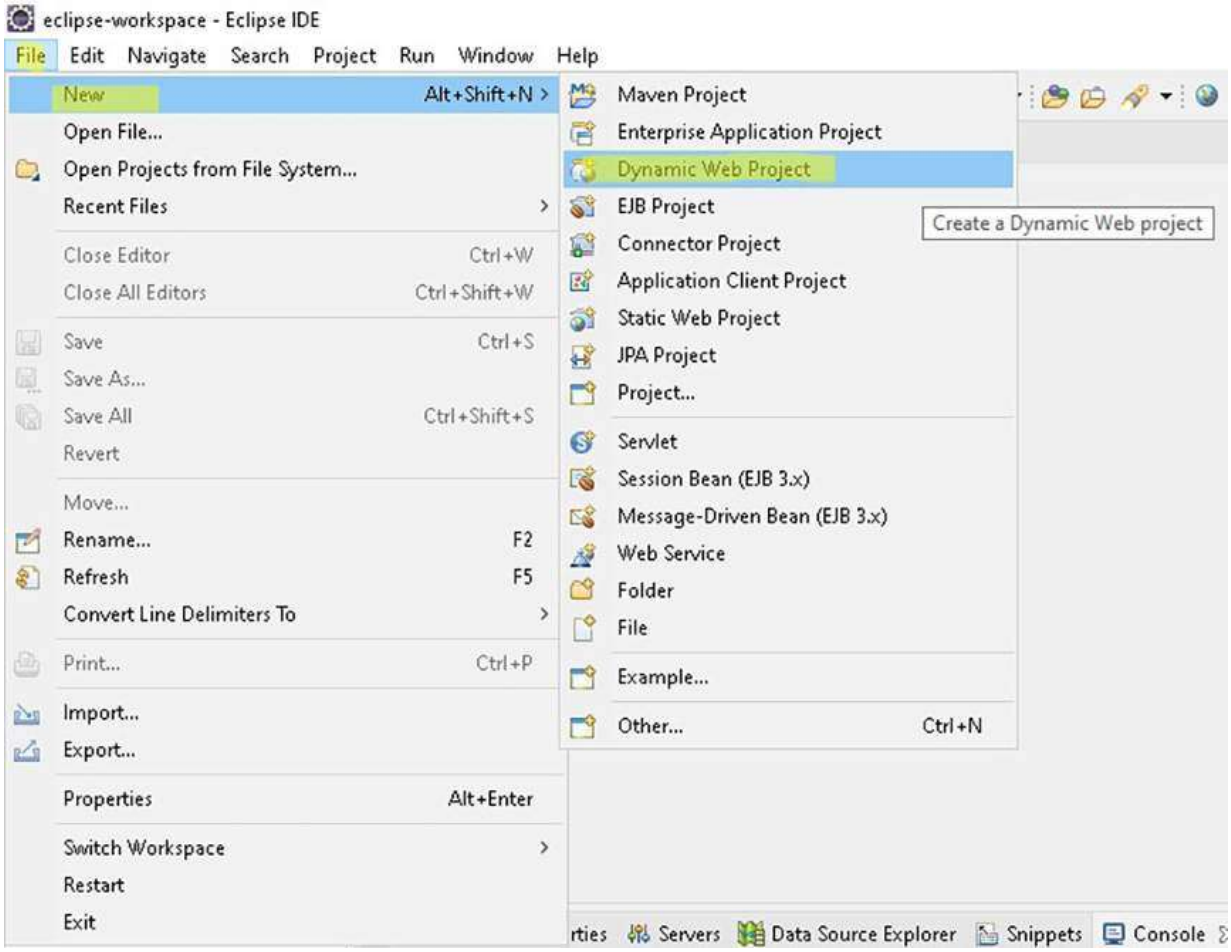


Figura 2
Fonte: Reprodução

Precisamos dar um nome ao projeto, como estamos em fase de aprendizado, eu decidi dar o nome de servletexemplo.

Na Figura a seguir, temos que selecionar algumas opções como *Target runtime*, nesse ponto podemos selecionar o server que foi instalado nas primeiras unidades (*Tomcat* 8.0 ou *Tomcat* 9.0).

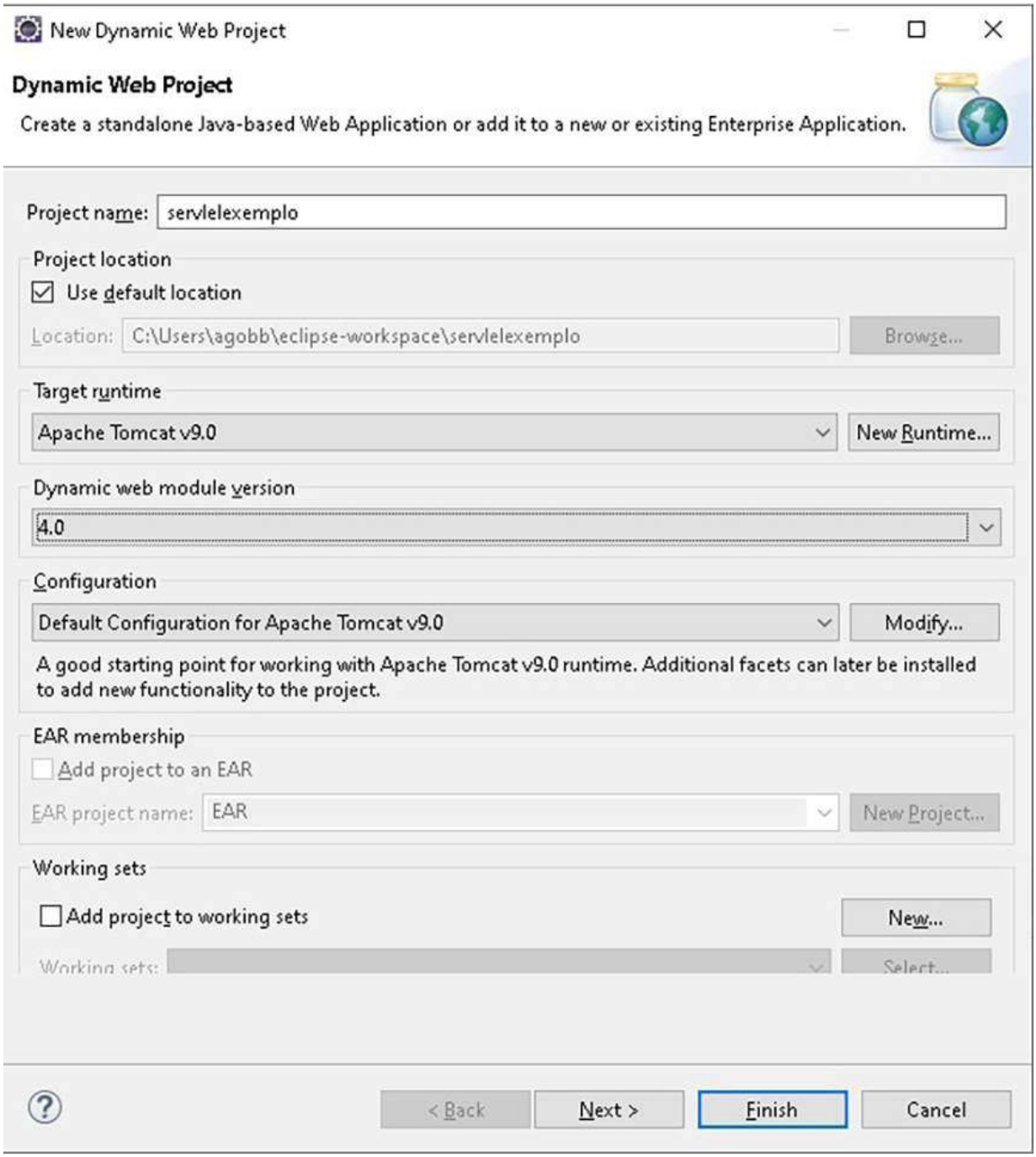


Figura 3
Fonte: Reprodução

Definimos a pasta onde os arquivos criados serão salvos.

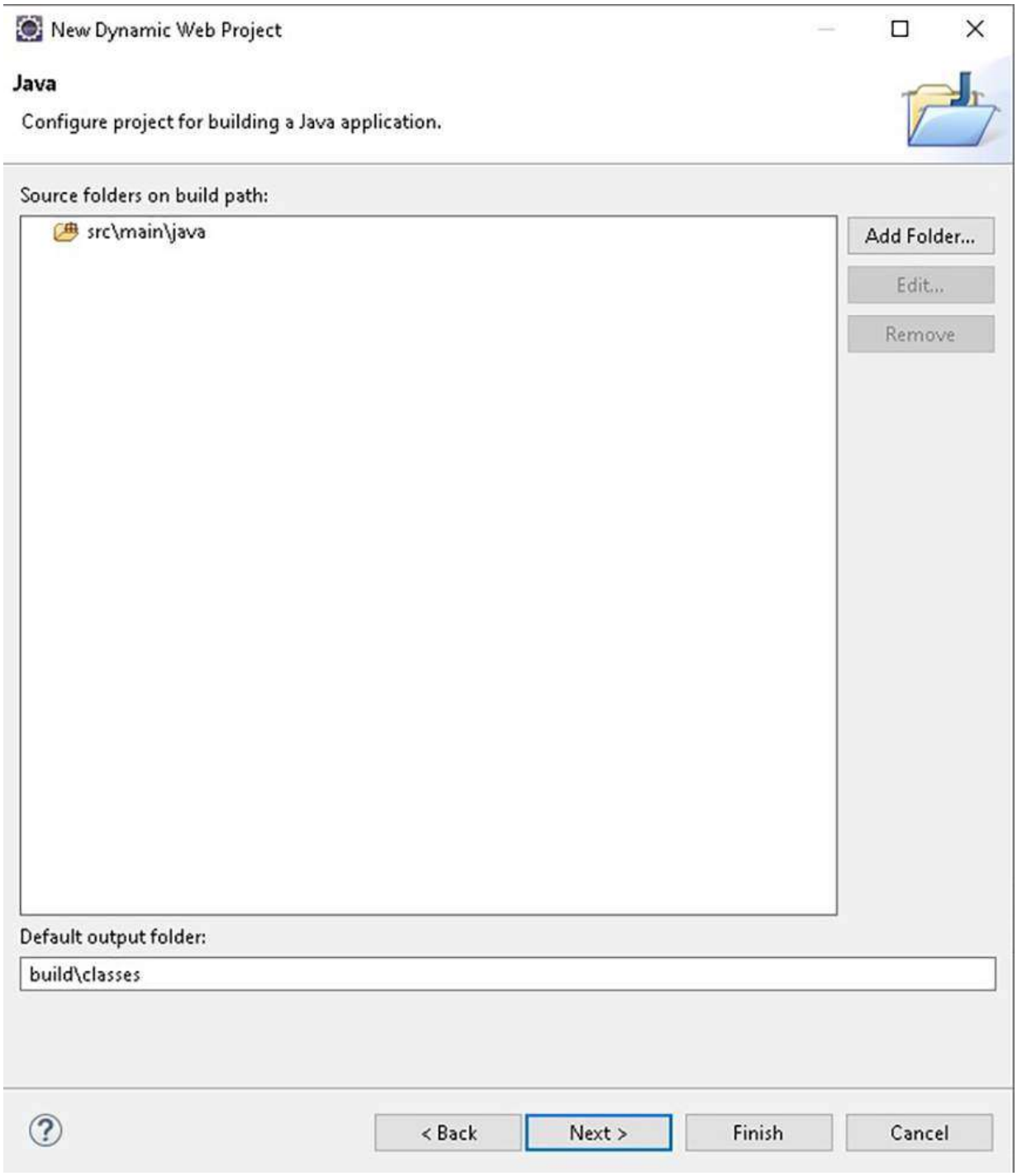


Figura 4
Fonte: Reprodução

Na Figura a seguir, podemos definir como o *servlet* será chamado no navegador ou por algum outro recurso, importante marcar a opção *Generate web.xml deployment descriptor*. Esse arquivo

web, será criado dentro da pasta *WEB-INF* e nele serão gravadas as informações de configurações como parâmetros para iniciar o projeto, apelidos e mapeamentos dos *servlets* etc.

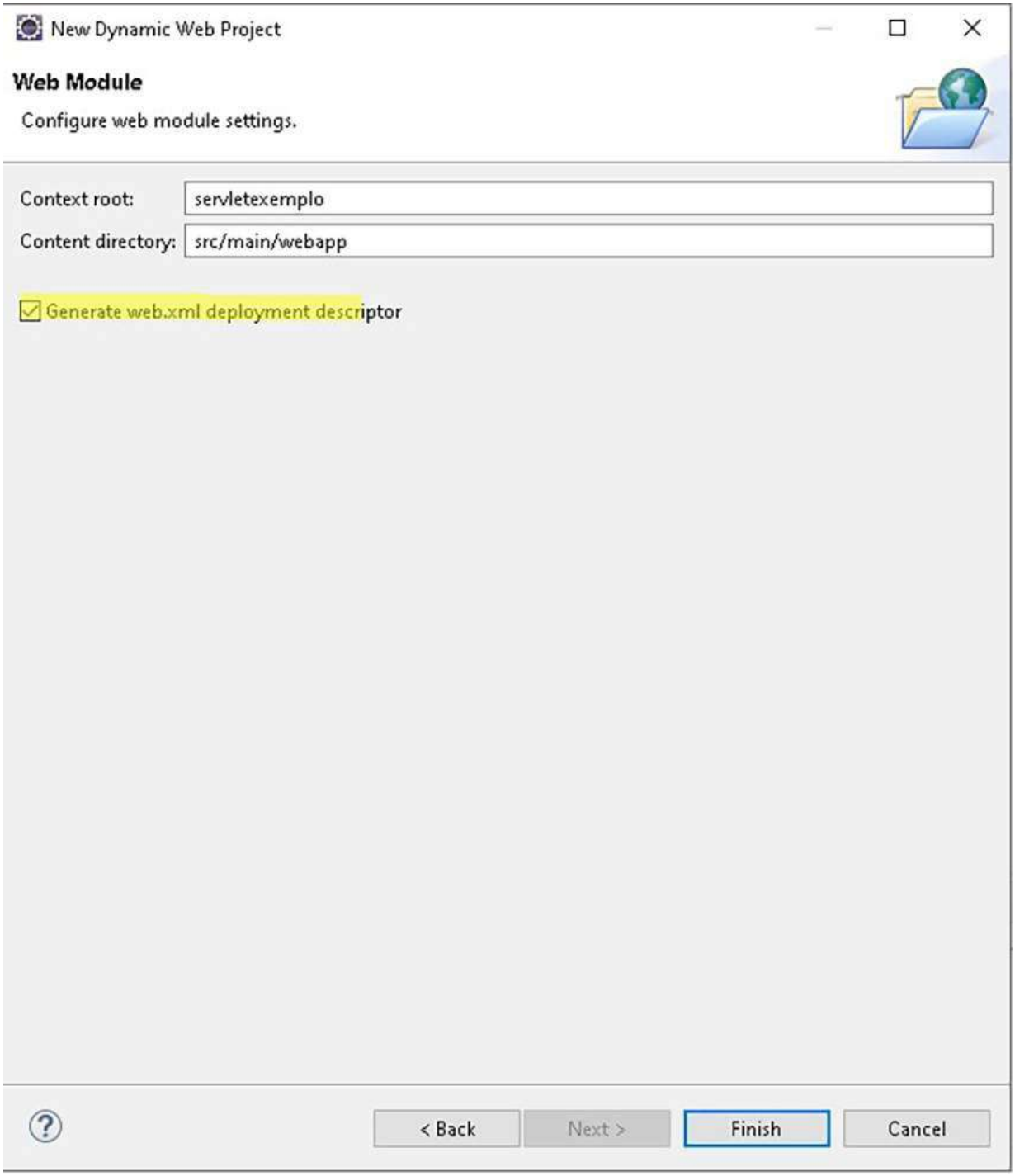


Figura 5
Fonte: Reprodução

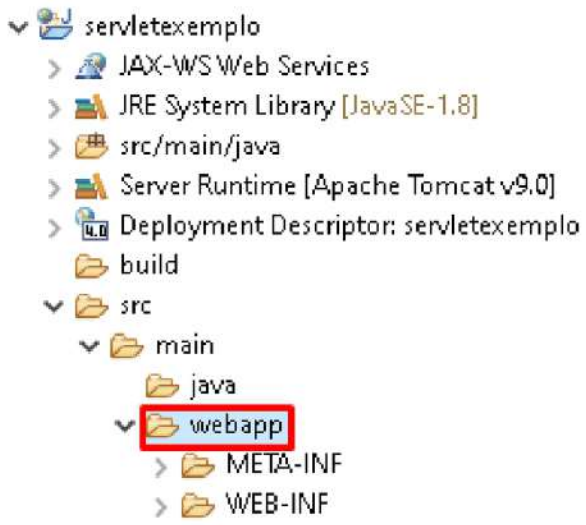


Figura 6
Fonte: Reprodução

Agora partimos para a parte em que criaremos o arquivo *servlet* em nosso projeto. Nesse primeiro momento, criaremos o arquivo na pasta *source*. O processo também é simples: escolher a opção *FILE* → *NEW* → *OTHER*..., procurar pela pasta *Web* e selecionar a opção *Servlet*.

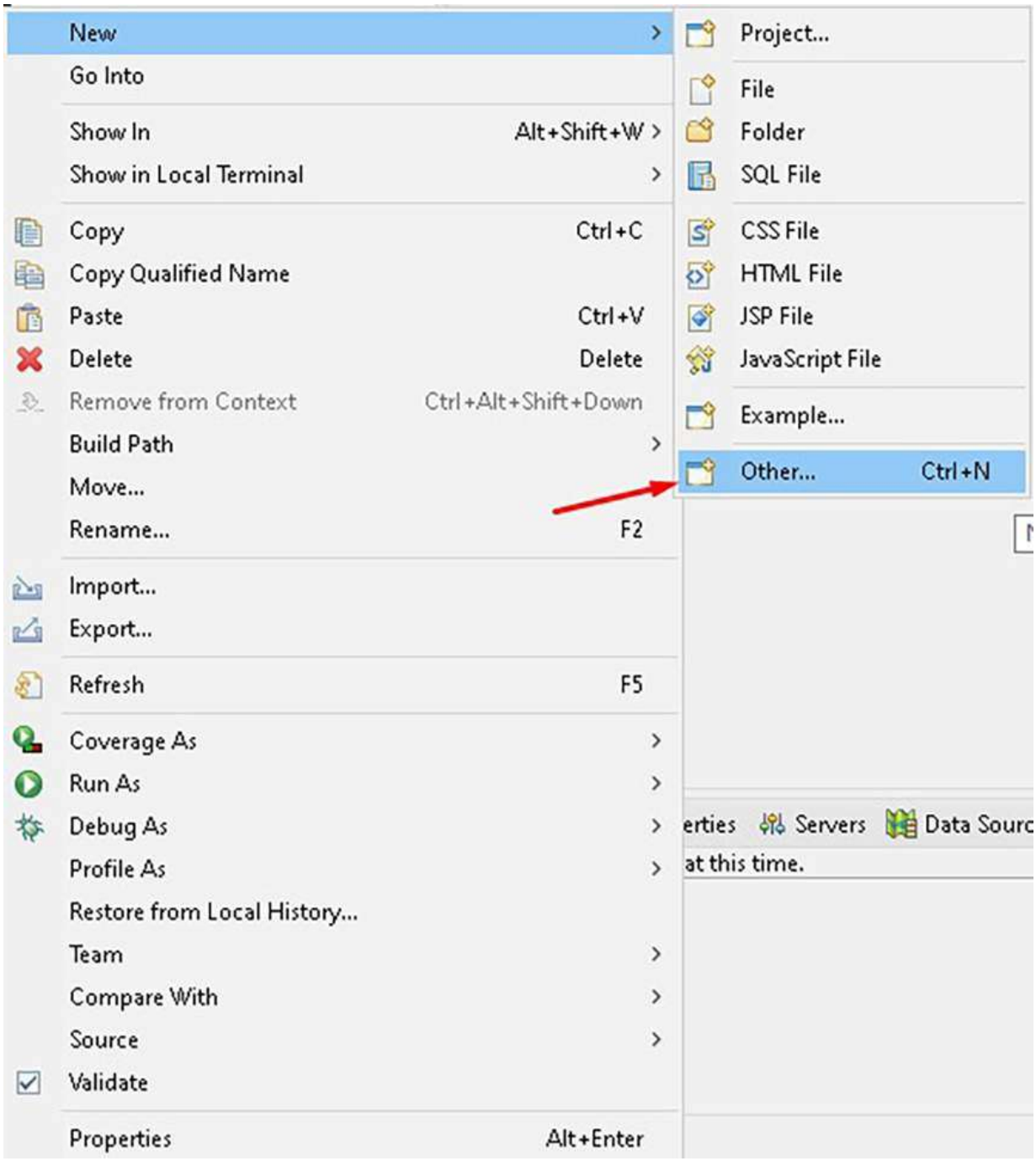


Figura 7
Fonte: Reprodução

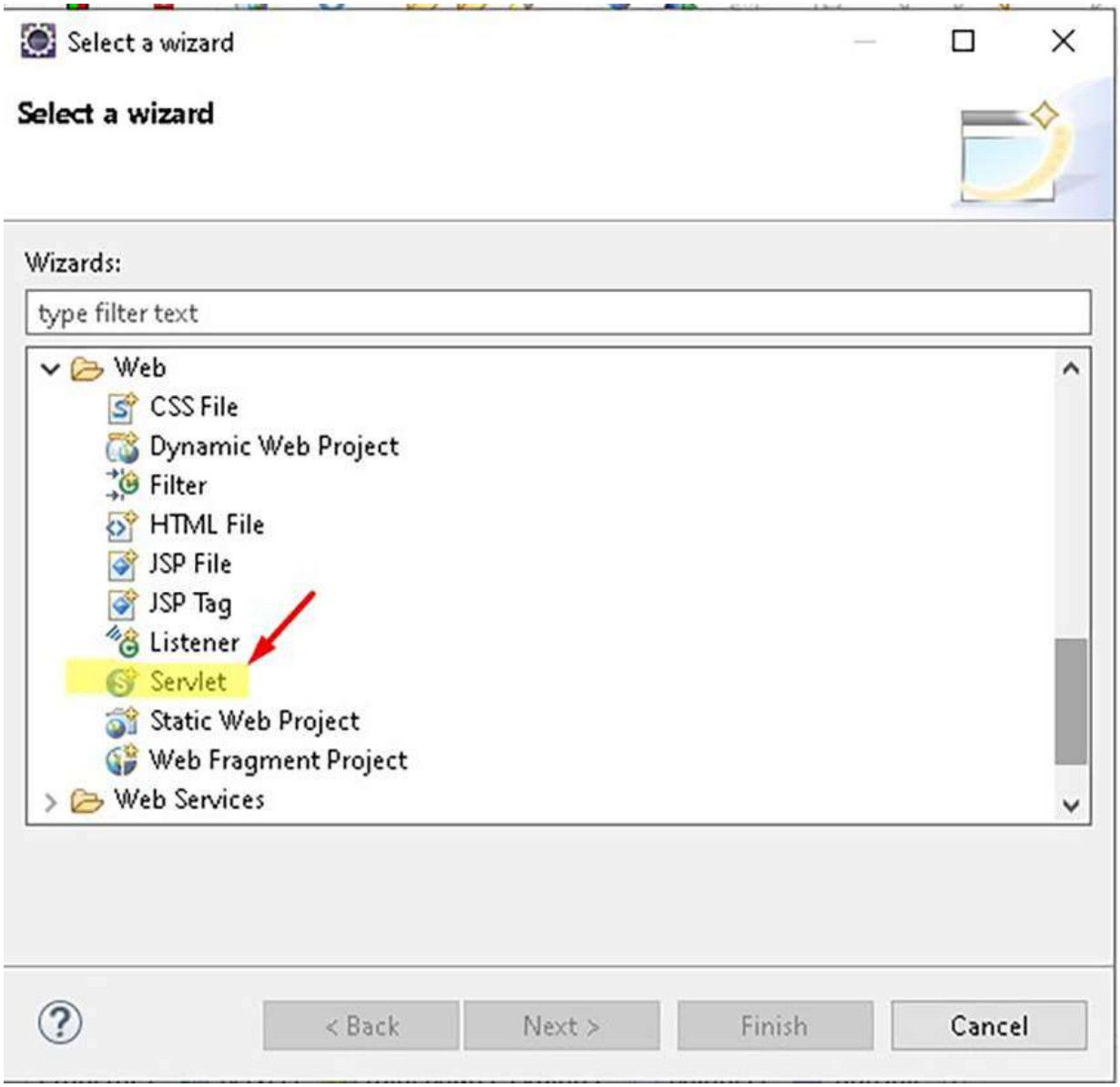


Figura 8
Fonte: Reprodução

Agora precisamos definir o nome do arquivo *servlet*, lembrando que o *servlet* é um arquivo com extensão *.java*. Nesse momento, podemos começar a utilizar o padrão de desenvolvimento e por isso criaremos um pacote (*package*) em que o arquivo será salvo.

Devemos acompanhar as Figuras a seguir:

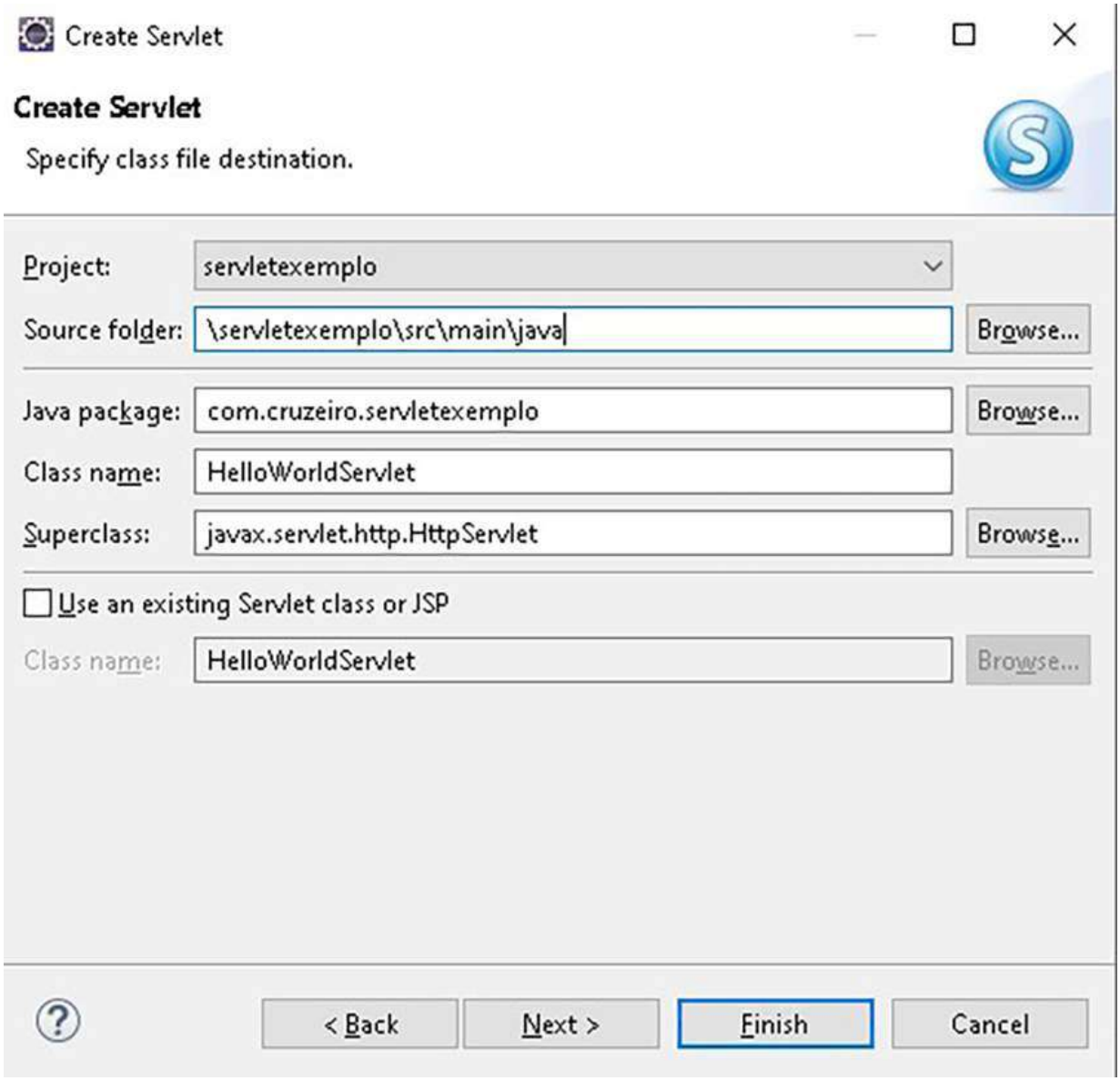


Figura 9
Fonte: Reprodução

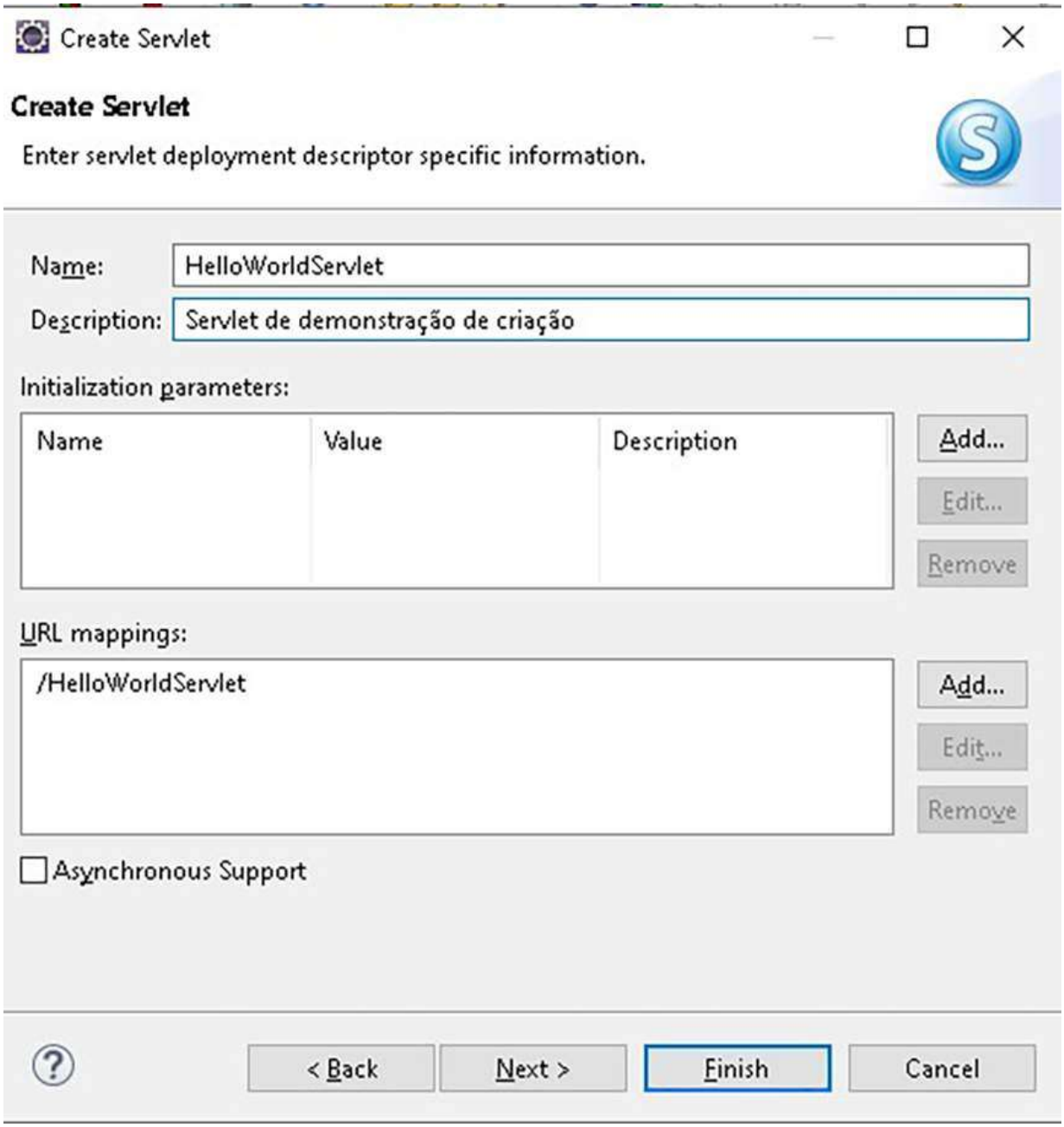


Figura 10
Fonte: Reprodução

Como estamos criando um *servlet*, geralmente ele pode receber requisições provenientes do *POST* ou *GET*. Importante então selecionarmos as opções que estão destacadas em amarelo conforme as próximas Figuras.

Você sabe o que é envio de dados via *POST* ou *GET*, não?!

Logo mais explicaremos a diferença.

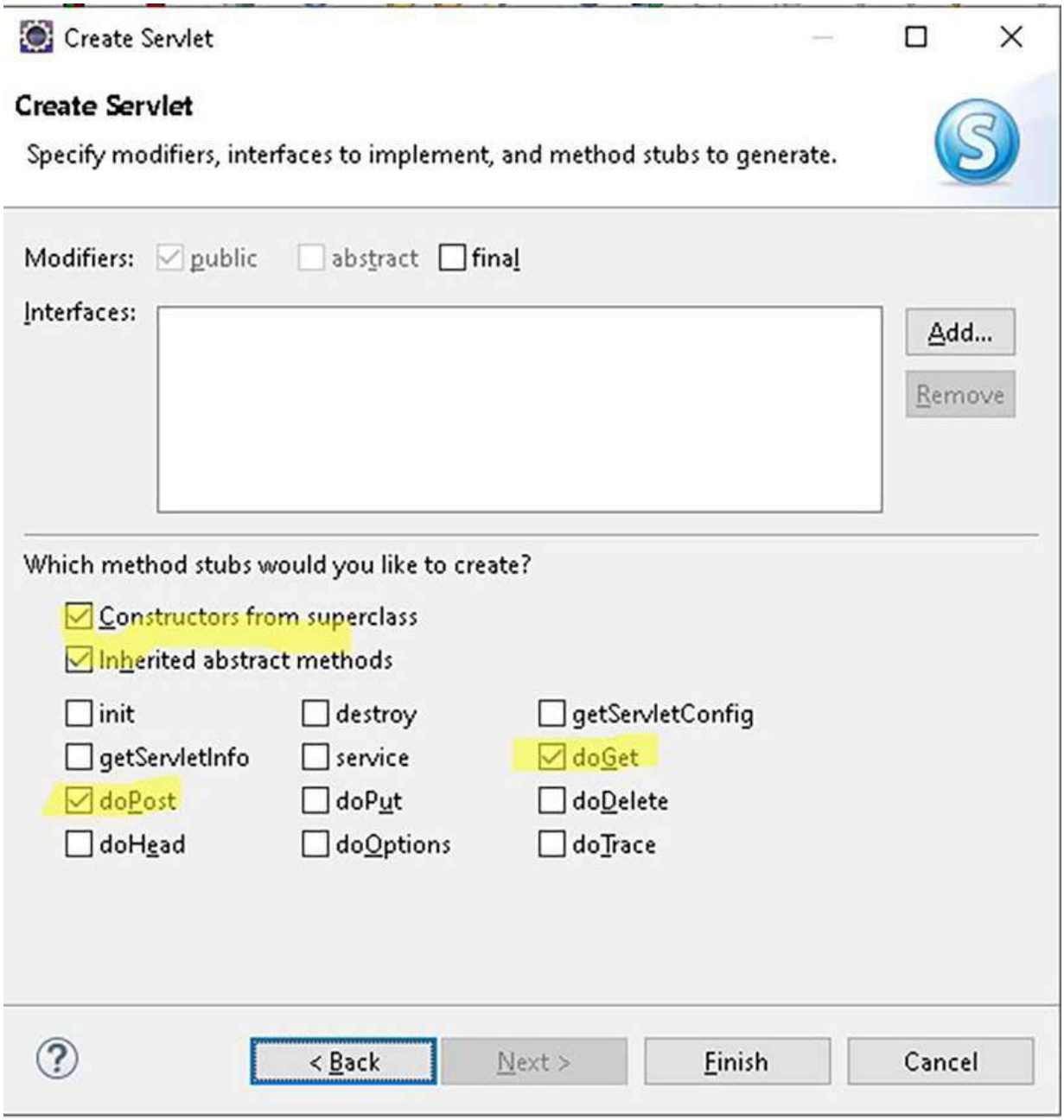


Figura 11
Fonte: Reprodução

Pronto, *servlet* criado e pronto para começarmos a programação.

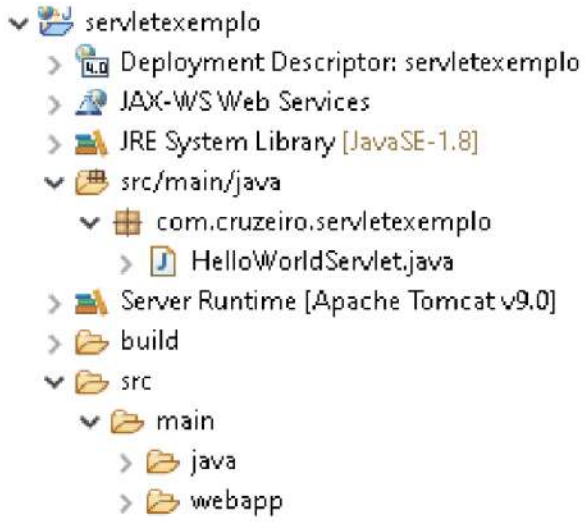


Figura 12
Fonte: Reprodução

Ao acessar o arquivo *servlet* criado, incialmente ele já vem com alguns códigos padrões como o construtor, os métodos *doGet* e *doPost*.

Ambos recebem como parâmetros objetos do tipo *request* e *response* para que possamos manipular as requisições e respostas.



Figura 13
Fonte: Reprodução

Vocês saberiam dizer qual a função do *servlet*?

Bom, vamos lá de uma forma direta e rápida explicar. *Servlet* é uma classe *java* que pode criar ou gerar outras páginas dinamicamente para a camada de apresentação das aplicações *web*. O *servlet* recebe as requisições HTTP (*Post* ou *Get*) que são processadas e então enviadas as respostas para quem as solicitou, os *servlets* trabalham muito bem em conjunto com a tecnologia JSP (*Java Server Pages*).

Então como exemplo, nas próximas imagens, mostraremos como colocar um código hmtl no *servlet* e então publicar e fazer com que o servidor *web tomcat* possa executar o *servlet* e transformá-lo em uma página na *web*.

```
 * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse
protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    // Passo 1: configurar o content type
    response.setContentType("text/html");

    // Passo 2: configurar o objeto para "escrever" no navegador
    PrintWriter out = response.getWriter();

    // Passo 3: gerar o html que será exibido no navegador
    out.println("<html><body>");

    out.println("<h2>Hello World</h2>");
    out.println("<hr>");
    out.println("Time on the server is: " + new java.util.Date());

    out.println("</body></html>");
}

 * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse
protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    // TODO Auto-generated method stub
    doGet(request, response);
}
```

Figura 14
Fonte: Reprodução

O *out.println* é um comando no *java* que irá "imprimir" na tela algo que foi passado para ele, esse é uma das formas de gerarmos um conteúdo dinâmico. Observem na imagem anterior que os códigos que foram usados são texto. Ao ser executado o *servlet* pelo *tomcat*, ele gera um arquivo html, onde o navegado (cliente) pode interpretar o texto e exibir o resultado.

Algumas bibliotecas precisam ser importantes, mas falaremos mais sobre isso no vídeo desta Unidade.

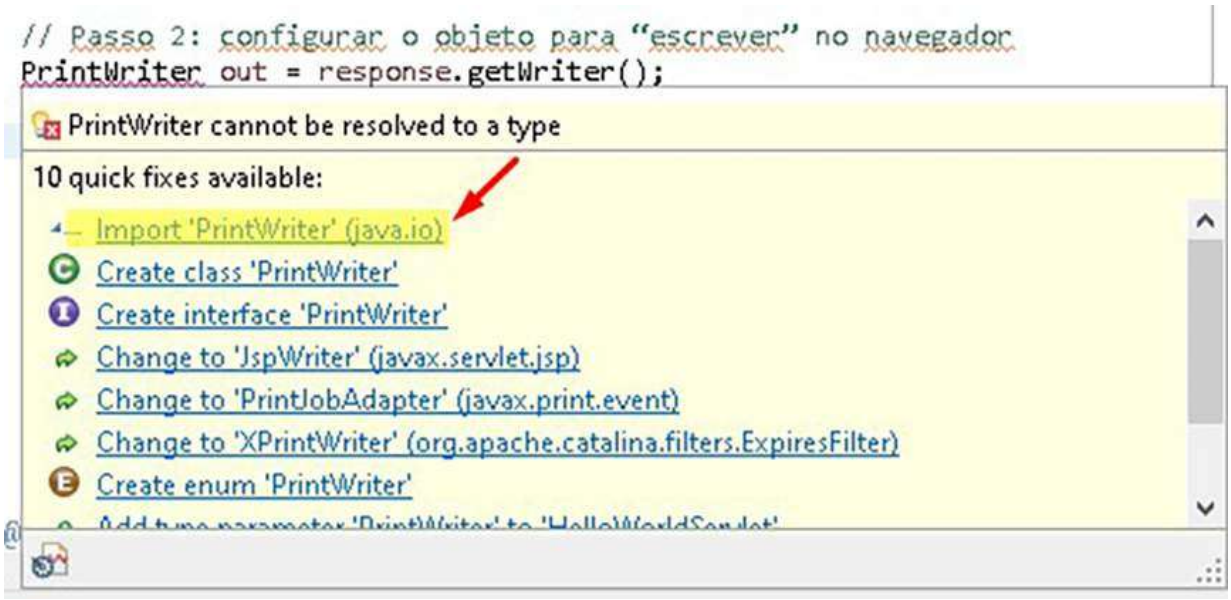


Figura 15
Fonte: Reprodução

Pronto, agora é compilar e executar o arquivo e ver o resultado no navegador.

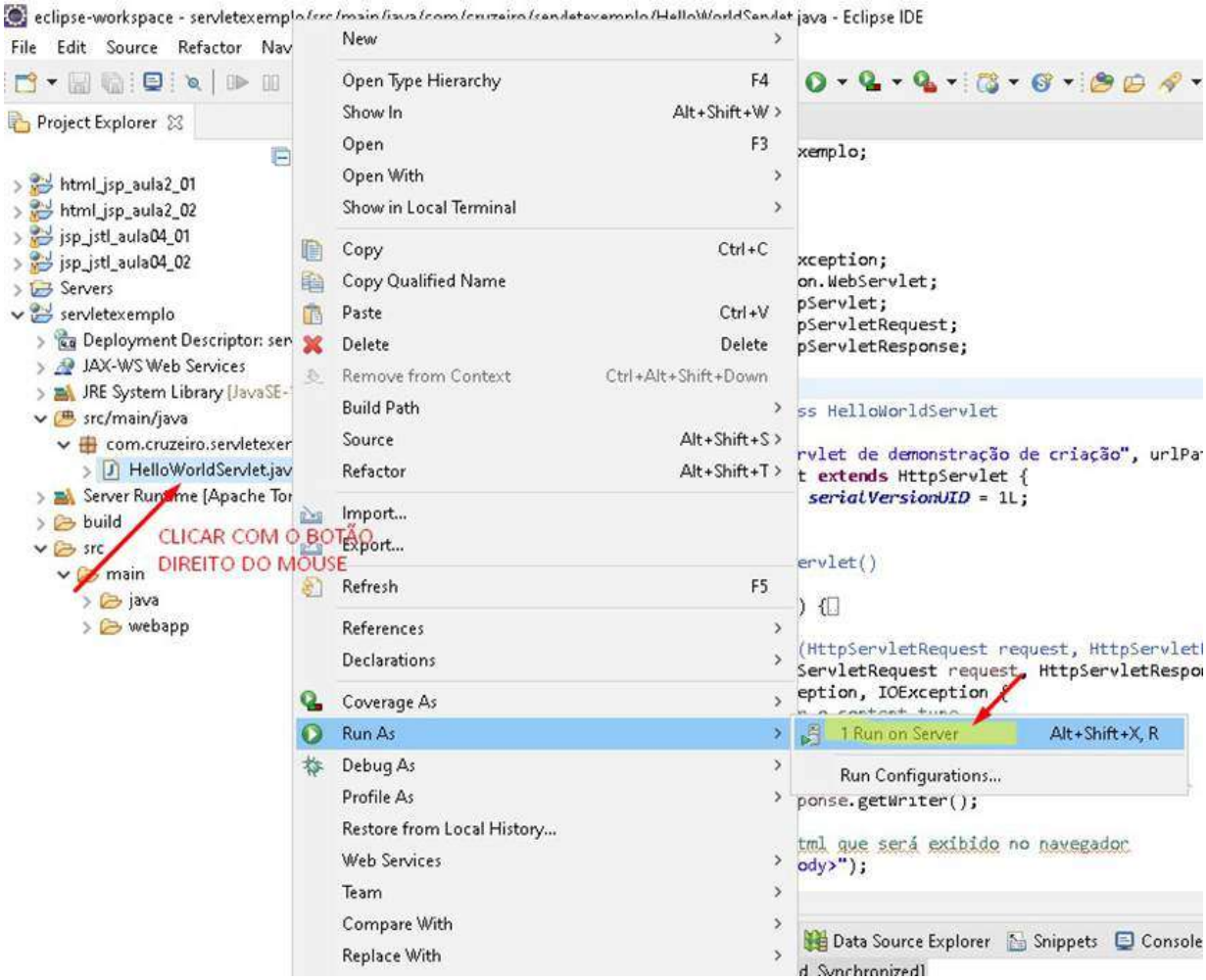


Figura 16
Fonte: Reprodução

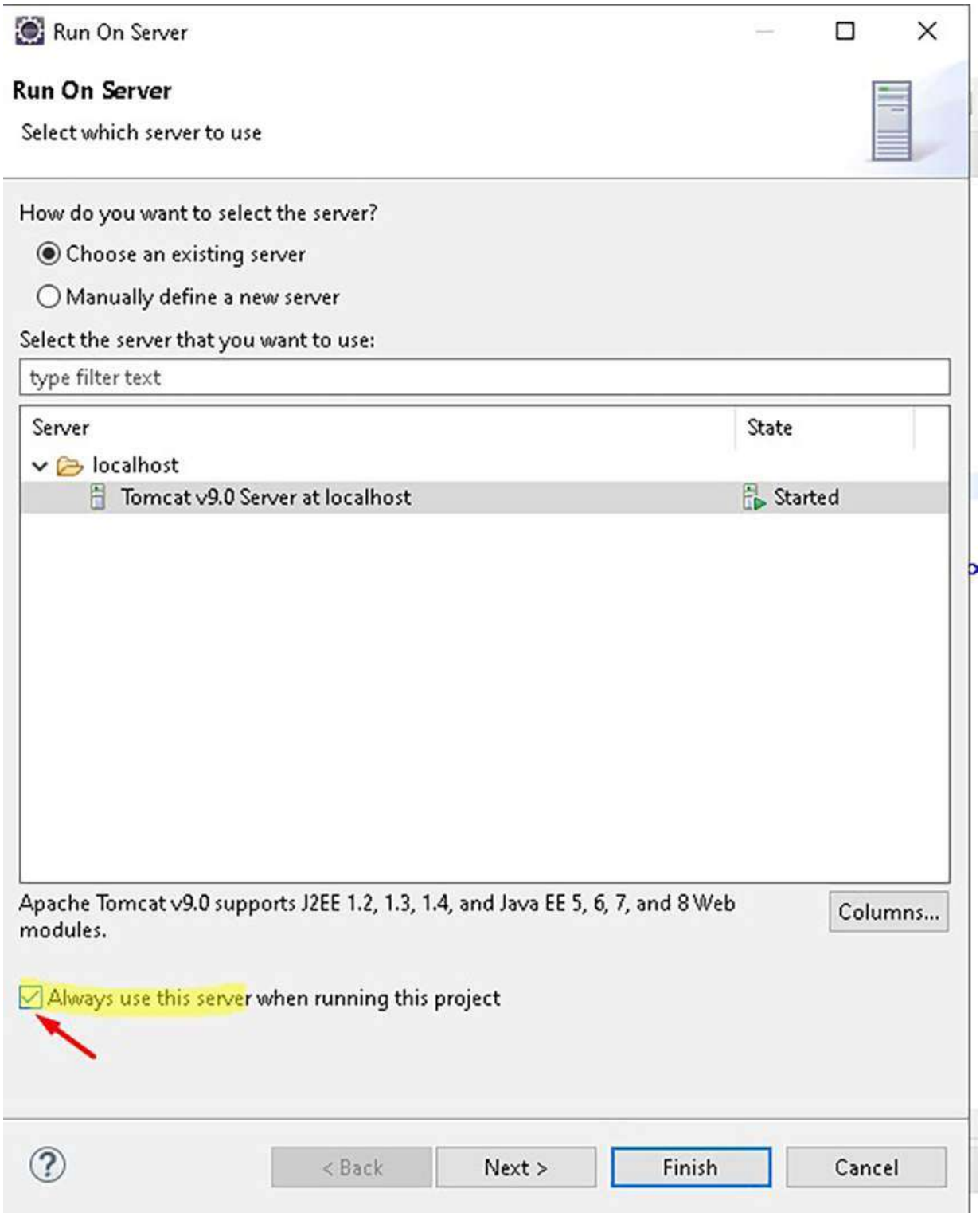


Figura 17
Fonte: Reprodução

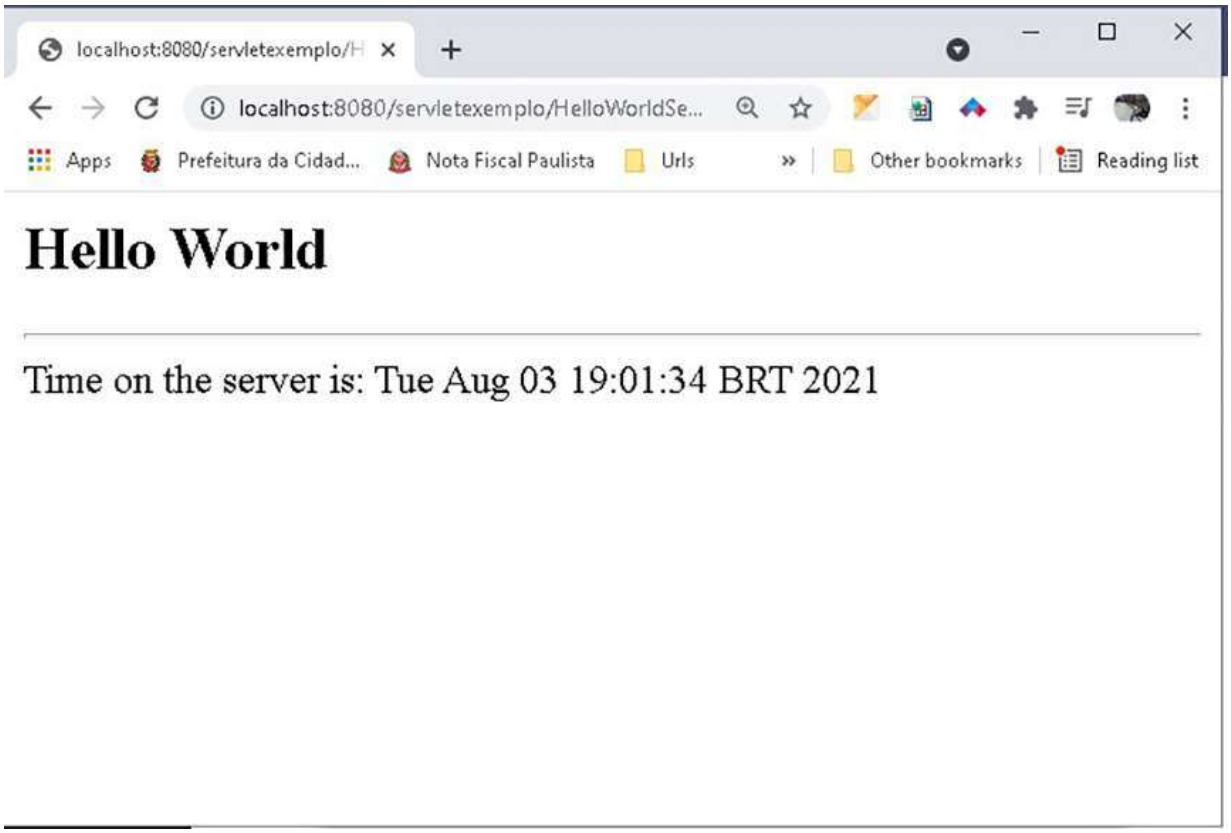


Figura 18
Fonte: Reprodução

Anteriormente falamos nas requisições HTTP (*POST* e *GET*).

Qual método usar e para que servem?

Método *GET*

- Excelente para debugar;
- Informações na url;
- Tamanho do conteúdo limitado.

Método *POST*

- Informações não visíveis na url;
- Sem tamanho do conteúdo limitado;
- Pode-se enviar informações do tipo binária.

Toda requisição ou vai ser enviada pelo método *GET* ou *POST*, por isso no *servlet* já existem os métodos padrões *doGet* e *doPost*, para que o programador possa colocar toda programação, regra de negócio nesses métodos.

Já que estamos aprendendo a tecnologia *Servlet* e já estudamos a tecnóloga JSP, podemos então entrar no assunto MVC.

Model-View-Controller (MVC) Design Pattern

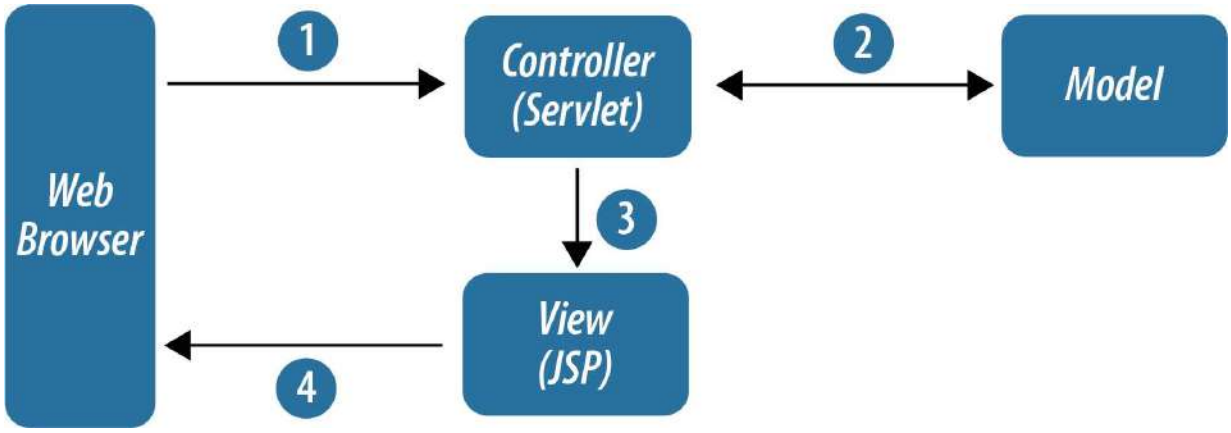


Figura 19

Benefícios:

- Diminuir as instruções HTML no código *Servlet*;
- Evitar o comando *out.println*(...) no *Servlet*;
- Minimizar o uso da lógica do negócio no JSP;
- Evitar o uso excessivo de *scriptles* no código JSP.

Como definido por Paz e Olsen (2013, pág. 8): O padrão mvc em si facilita a complexidade do gerenciador, separando claramente a funcionalidade do aplicativo em três partes centrais, o modelo, a visualização e o controlador.

A ideia é que possamos programar em camadas, ou seja, separar o que seria apresentação (páginas html, jsp e jstl), regra de negócio (*servlet*) e *model* (banco de dados)

Bom, finalizando então, com esses conceitos abordados estamos prontos para criarmos uma aplicação em três camadas.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Java para *Web*: Desenvolvimento de Aplicações

ALVES, W. P. **Java para *Web***: desenvolvimento de aplicações. São Paulo: Érica 2015.

Desenvolvendo Aplicações *Web* com JSP, *Servelts*, *Javaserver Faces*, *Hibernate*, EJB 3 *Persistance* e *Ajax*

GONÇALVES, E. **Desenvolvendo aplicações *Web* com JSP, *Servelts*, *Javaserver Faces*, *Hibernate*, EJB 3 *Persistance* e *Ajax***. Rio de Janeiro: Ciência Moderna, 2007.

Especificações Técnicas de *Software*

LIMA, A. S. **Especificações técnicas de *software***. São Paulo: Érica, 2012. 384 p.

Programando com asp.net mvc: Aprenda a Desenvolver Aplicações *Web* Utilizando a Arquitetura MVC

LOTAR, A. **Programando com asp.net MVC**: Aprenda a Desenvolver Aplicações *Web* Utilizando a Arquitetura MVC. São Paulo: Novatec, 2011. 392 p. ISBN 9788575222836.

Referências

SCHILDT, H. **Use a cabeça JSP e *Servlet***. 6. ed. Bookman, 2015.

PAZ, J. R. G.; OLSEN, A. ***Beginning* asp.net MVC 4**. Apress, 2013.